

1. УВОД И ПОСТАВКА ЗАДАТКА

1.1. Циљ задатка

Циљ овог задатка јесте пројектовање, рачунарска симулација и анализа рада дигиталних П (Пропорционалног) и ПИ (Пропорционално-Интегралног) регулатора у систему са затвореном повратном спрегом. Управљани објекат представља једносмерни електромотор (односно процес првог реда) дефинисан континуалном преносном функцијом првог реда која износи:

$$G(s) = \frac{K}{\tau s + 1} = \frac{0.8}{0.25s + 1}$$

Где је $K = 0.8$ статичко појачање објекта, а $\tau = 0.25$ [s] временска константа система. Кроз практичну имплементацију у окружењу Siemens TIA Portal (SCL) и визуелизацију на HMI Trend View графику, циљ је верификовати следеће објективне показатеље квалитета управљања:

- Поређење П и ПИ режима: Анализа утицаја пропорционалног појачања K_c на брзину одзива и доказивање присуства статичке грешке у стационарном стању при примени само П регулатора.
- Елиминација статичке грешке: Показивање како увођење интегралног дејства (K_i) потискује стационарну грешку на нулу.
- Спречавање засићења интегратора (Anti-Reset Windup): Практична верификација алгорита за ограничење интегралне суме у условима засићења (односно сатурације) управљачког сигнала.

1.2. Одређивање почетних параметара и дискретизација

Да би се континуални алгоритам могао реализовао у PLC управљачком блоку OB30 (Cyclic Interrupt) са периодом одабирања $T = 0.04$ [s] (односно 40 [ms]), извршена је дискретизација објекта и прорачун параметара регулатора.

Применом Ојлерове методе (или аналитичким решењем диференцијалне једначине за период $T = 0.04$ [s] и $\tau = 0.25$ [s], добија се релација којом се у SCL коду у реалном времену израчунава излаз објекта $y[k]$, на следећи начин:

$$y[k] = e^{-\frac{T}{\tau}} \cdot y[k-1] + K \left(1 - e^{-\frac{T}{\tau}}\right) \cdot u[k]$$

$$y[k] = 0.86207 \cdot y[k-1] + 0.11034 \cdot u[k]$$

Знајући ово, могуће је касније модификовати модел електромотора мењајући вредности K и τ , чиме би смо успорили или убрзали систем, дајући нам другачије облике криве на графику, са обзиром на промену брзине система.

Подешавање параметара ПИ регулатора спроведено је методом поништавања доминантног пола објекта (pole-zero cancellation), где се интегрална временска константа регулатора T_i изједначава са временском константом објекта τ , односно:

$$T_i = \tau = 0.25 \text{ s}$$

На основу релације за континуално интегрално појачање $K_i = K_c / T_i$, и за усвојено пропорционално појачање у ПИ режиму $K_c = 3$, израчунава се континуално интегрално појачање:

$$K_i = \frac{K_c}{\tau} = \frac{3.0}{0.25 \text{ s}} = 12.0 \text{ s}^{-1}$$

Пошто се у дискретном SCL коду интегрална сума ажурира у сваком кораку одабирања $T = 0.04 \text{ [s]}$, израчунава се дискретни корак интеграције (дефинисан у коду као K_{i_step}) као:

$$K_{i_step} = K_i \cdot T = 12.0 \cdot 0.04 = \mathbf{0.48}$$

Са одређеним почетним параметрима можемо да кренемо на израду самог SCL кода у TIA Portal-у.

Будући да је овај рад заправо наставак задатка 1, као коришћени контролер се поново бира Simatic S7-1200 CPU, и то тачније модел CPU 1214C DC/DC/DC, верзије 4.4.

2. ДЕФИНИСАЊЕ И АНАЛИЗА PLC ПРОМЕНЉИВИХ (PLC TAGS)

За исправно функционисање математичког модела, алгоритама управљања и комуникације са HMI панелом, дефинисана је табела глобалних PLC променљивих (Default tag table). Све променљиве су Real типа (осим управљачких битова) како би се омогућила прецизна математика са покретним зарезом.

Комплетирана табела PLC тагова јесте следећа табела:

konacno ▸ PLC_1 [CPU 1214C DC/DC/DC] ▸ PLC tags ▸ Default tag table [41]								
Default tag table								
	Name	Data type	Address	Retain	Acces...	Writa...	Visibl...	Commer
1	w_setpoint	Real	%MD10	☐	☒	☒	☒	
2	y_objekat	Real	%MD14	☐	☒	☒	☒	
3	u_upravljanje	Real	%MD18	☐	☒	☒	☒	
4	e_greska	Real	%MD30	☐	☒	☒	☒	
5	u_integral	Real	%MD22	☐	☒	☒	☒	
6	mode_PI	Bool	%M0.1	☐	☒	☒	☒	
7	motor_on	Bool	%M0.0	☐	☒	☒	☒	
8	u_proporcionalni	Real	%MD26	☐	☒	☒	☒	
9	Kc_P	Real	%MD34	☐	☒	☒	☒	
10	Kc_PI	Real	%MD38	☐	☒	☒	☒	
11	Ki_step	Real	%MD42	☐	☒	☒	☒	
12	<Add new>			☐	☒	☒	☒	

w_setpoint - Задата вредност процесне величине (брзине) у процентима (0–100%)

y_objekat – Тренутна вредност излаза објекта (односно стварна брзина мотора)

u_upravljanje – Укупни излазни сигнал из регулатора који се шаље мотору

e_greska - Сигнал грешке ($e = w - y$), односно одступање излаза од задате вредности

u_integral - Акумулирана вредност интегралне суме PI регулатора

mode_PI - Прекидач за избор режима рада: FALSE = P режим, TRUE = PI режим

motor_on - Главни прекидач за укључивање и ресетовање симулације погона

u_proporcionalni - Пропорционална компонента управљачког сигнала ($K_c * e$)

Kc_P - Пропорционално појачање регулатора у чистом P режиму

Kc_PI - Пропорционално појачање регулатора када је активан PI режим

Ki_step - Дискретни корак интеграције за период $T = 0.04$ [s] ($K_i * T$)

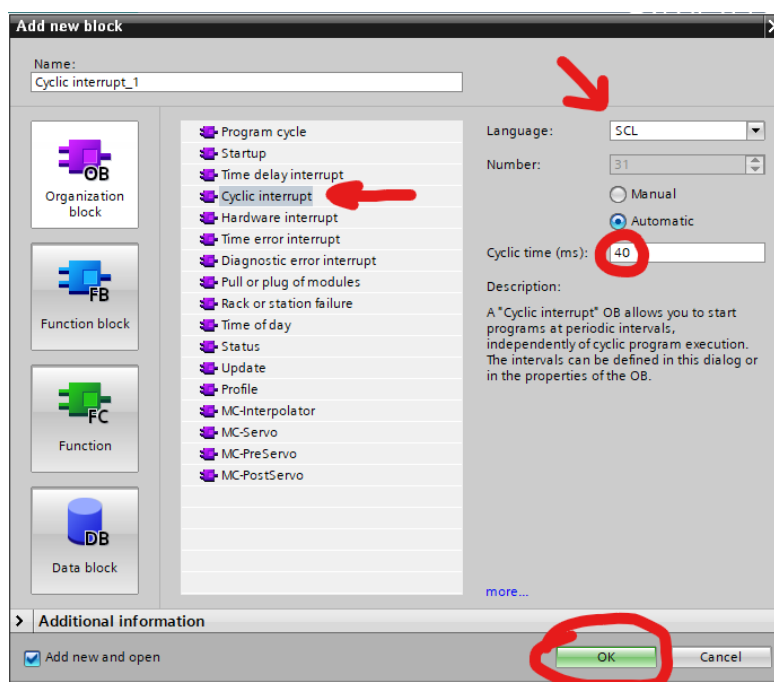
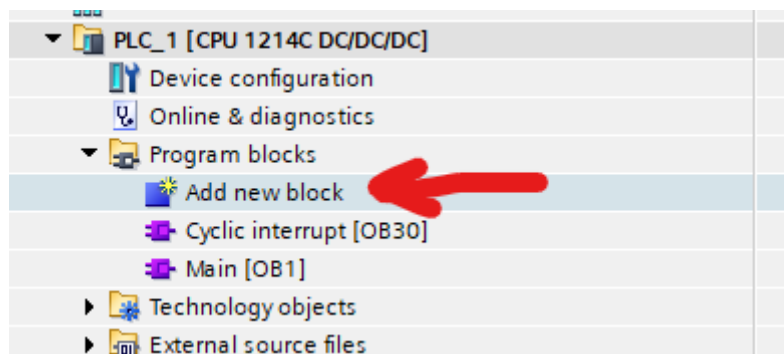
Како би детаљно објаснили улоге променљивих, можемо их сврстати у четири групе:

- Управљачки битови (`motor_on`, `mode_PI`), служе за избор режима рада и управљање симулацијом. `motor_on` омогућава ресетовање свих променљивих на 0 када је мотор искључен, док `mode_PI` преусмерава извршавање SCL кода између П и ПИ алгоритма.
- Улазне и излазне процесне променљиве (`w_setpoint`, `y_objekat`, `e_greska`), чине основну петљу са повратном спрегом. У сваком циклусу од 40 [ms], сигнал грешке `e_greska` се прерачунава као разлика задате вредности `w_setpoint` и тренутног одзива мотора `y_objekat`.
- Компоненте управљања (`u_proporcionalni`, `u_integral`, `u_upravljanje`), `u_proporcionalni` представља тренутно дејство сразмерно грешци, док `u_integral` акумулира претходна одступања ради елиминације статичке грешке. `u_upravljanje` формира коначни сигнал који делује на дискретни модел мотора.
- Параметри регулатора (`Kc_P`, `Kc_PI`, `Ki_step`) нам омогућавају независно подешавање појачања за различите режиме рада. Раздвајањем K_c за П и PI режим омогућено нам је брзо упоређивање резултата у Watch tabeli без измене самог SCL кода.

3. ИМПЛЕМЕНТАЦИЈА У СИСТЕМУ TIA PORTAL (SCL)

Да би се обезбедило стриктно временско извршавање управљачког алгоритма и модела објекта, код се не пише у стандардном Main [OB1] блоку, већ у прекидном организационом блоку.

- Креирање OB30 блока: У “Project tree”-у са леве стране, под одабраним PLC-ом, изабере се “Add new block” па “Organization block (OB)” и затим се селекује из листе “Cyclic interrupt (OB30)”.
- Додатно, потребно је подесити задату периоду одабирања, односно за “Cyclic time” се уместо 100 уписује 40 [ms] (због услова да је $T = 0.04$ [s]). Такође је потребно изабрати тачан програмски језик при креирању OB30 (SCL - Structured Control Language).



У овако креиран програмски блок убацујемо следећи SCL код:

```
IF "motor_on" THEN

    // Izracunavanje greske upravljanja
    "e_greska" := "w_setpoint" - "y_objekat";

    // Algoritam regulatora
    IF "mode_PI" THEN
        "u_integral" := "u_integral" + ("Ki_step" * "e_greska");

        // ARW - Ogranicavanje integratora na opseg (0 - 125)
        IF "u_integral" > 125.0 THEN
            "u_integral" := 125.0;
        ELSIF "u_integral" < 0.0 THEN
            "u_integral" := 0.0;
        END_IF;

        "u_upravljanje" := ("Kc_PI" * "e_greska") + "u_integral";
    ELSE
        // P Regulator sa promenljivim Kc_P
        "u_integral" := 0.0;
        "u_upravljanje" := "Kc_P" * "e_greska";
    END_IF;

    // Simulacija objekta prvog reda
    "y_objekat" := (0.86207 * "y_objekat") + (0.11034 * "u_upravljanje");

    // Fizicko ograničenje izlaza motora (0 - 100)%
    IF "y_objekat" > 100.0 THEN
        "y_objekat" := 100.0;
```

```
ELSIF "y_objekat" < 0.0 THEN  
    "y_objekat" := 0.0;  
END_IF;
```

```
ELSE
```

```
    // Reset regulatora pri gasenju  
    "u_integral" := 0.0;  
    "u_upravljanje" := 0.0;  
    "y_objekat" := 0.0;  
    "e_greska" := 0.0;  
END_IF;
```

Сам код је структуриран тако да целокупну логику управљања и симулације условљава главним прекидачем рада (`motor_on`), односно мотор даје вредности рада само када је укључен.

- Главна условна грана (`motor_on = TRUE`):

У сваком циклусу од 40 [ms] прво се формира грешка управљања као разлика задате вредности и тренутног одзива мотора (`"e_greska" := "w_setpoint" - "y_objekat"`).

- Избор режима (`mode_PI`):

Када је `mode_PI = TRUE`, активира се ПИ режим. Интегратор акумулира суму преко дискретног корака `Ki_step`, уз примењену Anti-Reset Windup (ARW) логику која ограничава интегралну суму на опсег од 0.0 до 125.0, како би се спречило препуњавање интегратора и успоравање система.

Укупно управљање представља збир П и И компоненте. Када је `mode_PI = FALSE`, ради само П регулатор са појачањем `Ks_P`, док се интегрална сума ресетује и држи на 0.

- Дискретна симулација мотора:

На основу прорачунатог управљања `u_upravljanje`, израчунава се нова брзина мотора за текући дискретни тренутак према једначини $y[k] = 0.86207 * y[k-1] + 0.11034 * u[k]$.

- Ограничење физичког излаза мотора:

Спречава се да прорачунати излаз мотора изађе из опсега (0.0, 100.0)%, чиме се симулирају реална физичка ограничења мотора (мотор не може да ради више од свог максимума од 100 %).

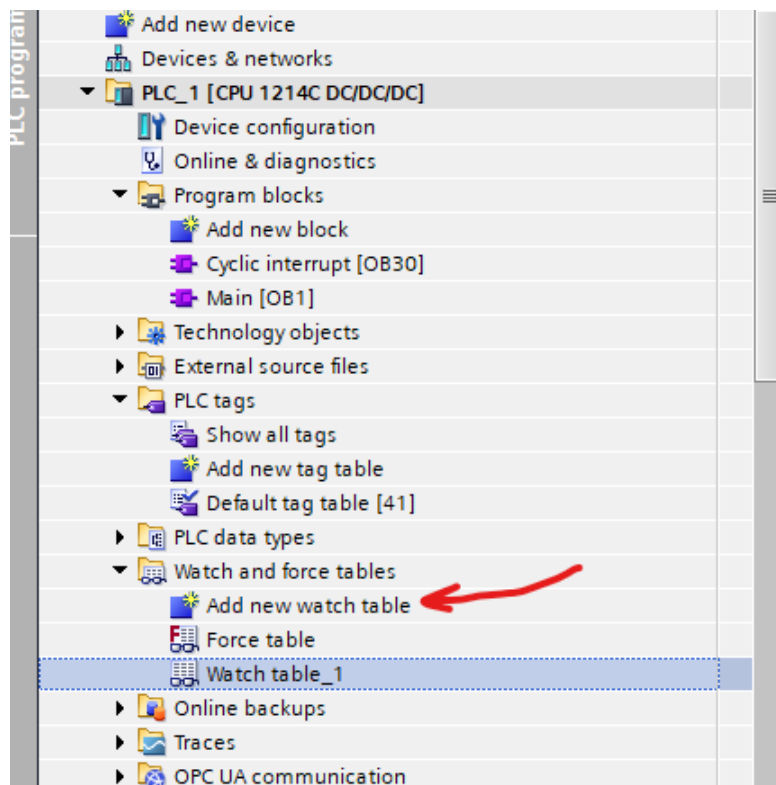
- На самом крају кода налази се сигурносни ресет (`motor_on = FALSE`):

Уколико је мотор искључен, све кључне променљиве (`u_integral`, `u_upravljanje`, `y_objekat`, `e_greska`) моментално се постављају на 0.0. Ово спречава неочекиване прелазне процесе и обезбеђује да свако ново покретање креће из дефинисаног почетног стања.

4. КРЕИРАЊЕ И КОРИШЋЕЊЕ WATCH ТАБЕЛЕ (WATCH TABLE)

За потребе тестирања, симулације и дијагностике рада алгоритма у реалном времену, креирана је надзорна табела Watch table_1. Она нам омогућава праћење тренутних вредности свих променљивих (преко функције Monitor value), као и измену параметара у току рада контролера (преко функције Modify value).

Да би смо користили ове функције потребно је прво да се изврши креирање саме табеле а то се врши одабиром опције са Project tree-а са леве стране, под опцијом Watch and force tables, Add new watch table.



У овако добијену табелу уносимо променљиве чије вредности желимо да пратимо или мењамо током симулације. Пример попуњене табеле је дат испод:

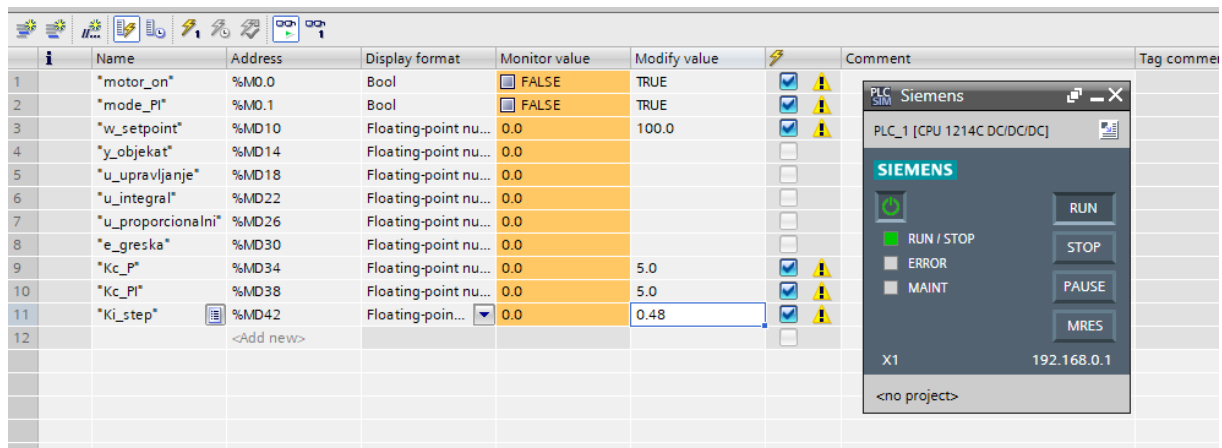
	Name	Address	Display format	Monitor value	Modify value		Comment
1	"motor_on"	%M0.0	Bool			☐	
2	"mode_PI"	%M0.1	Bool			☐	
3	"w_setpoint"	%MD10	Floating-point nu...			☐	
4	"y_objekat"	%MD14	Floating-point nu...			☐	
5	"u_upravljanje"	%MD18	Floating-point nu...			☐	
6	"u_integral"	%MD22	Floating-point nu...			☐	
7	"u_proporcionalni"	%MD26	Floating-point nu...			☐	
8	"e_greska"	%MD30	Floating-point nu...			☐	
9	"Kc_P"	%MD34	Floating-point nu...			☐	
10	"Kc_PI"	%MD38	Floating-point nu...			☐	
11	"Ki_step"	%MD42	Floating-poi...			☐	
12		<Add new>				☐	

Пројектни задатак – аутоматско управљање

При попуњавању табеле важно је да се имена променљивих повежу са одговарајућим PLC таговима из табеле тагова, као и да се за управљачке битове подеси Bool дисплеј формат.

Када се покрене симулација нашег PLC-а, у Watch табели се може укључити функција Monitor all (иконица са наочарима са зеленим троуглићем испод ње) која ће нам дати резултате симулације у реалном времену.

Ако желимо да модификујемо вредности променљивих, довољно је да их упишемо у колону Modify value, и затим је ОБАВЕЗНО да се кликне дугме за Modify all selected values once and now (иконица која изгледа као жута муњница са бројем један поред). Ако се не кликне дугме за само модификовање вредности, исто је као да ништа нисмо ни укуцали.



	Name	Address	Display format	Monitor value	Modify value		Comment	Tag comment
1	"motor_on"	%M0.0	Bool	FALSE	TRUE			
2	"mode_PI"	%M0.1	Bool	FALSE	TRUE			
3	"w_setpoint"	%MD10	Floating-point nu...	0.0	100.0			
4	"y_objekat"	%MD14	Floating-point nu...	0.0				
5	"u_upravljanje"	%MD18	Floating-point nu...	0.0				
6	"u_integral"	%MD22	Floating-point nu...	0.0				
7	"u_proporcionalni"	%MD26	Floating-point nu...	0.0				
8	"e_greska"	%MD30	Floating-point nu...	0.0				
9	"Kc_P"	%MD34	Floating-point nu...	0.0	5.0			
10	"Kc_PI"	%MD38	Floating-point nu...	0.0	5.0			
11	"Ki_step"	%MD42	Floating-point nu...	0.0	0.48			
12		<Add new>						

PLC Siemens

PLC_1 [CPU 1214C DC/DC/DC]

SIEMENS

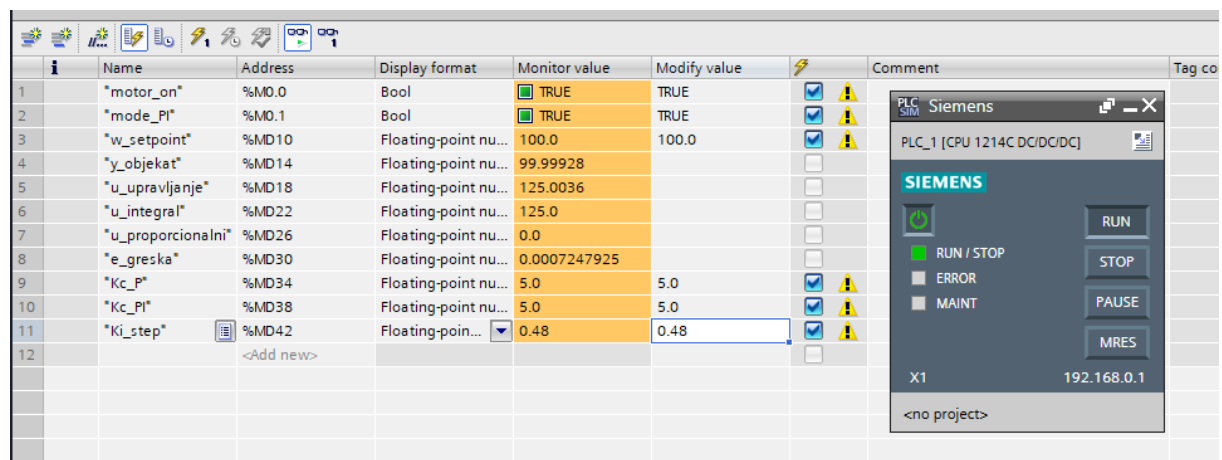
RUN / STOP STOP

ERROR PAUSE

MAINT MRES

X1 192.168.0.1

<no project>



	Name	Address	Display format	Monitor value	Modify value		Comment	Tag comment
1	"motor_on"	%M0.0	Bool	TRUE	TRUE			
2	"mode_PI"	%M0.1	Bool	TRUE	TRUE			
3	"w_setpoint"	%MD10	Floating-point nu...	100.0	100.0			
4	"y_objekat"	%MD14	Floating-point nu...	99.99928				
5	"u_upravljanje"	%MD18	Floating-point nu...	125.0036				
6	"u_integral"	%MD22	Floating-point nu...	125.0				
7	"u_proporcionalni"	%MD26	Floating-point nu...	0.0				
8	"e_greska"	%MD30	Floating-point nu...	0.0007247925				
9	"Kc_P"	%MD34	Floating-point nu...	5.0	5.0			
10	"Kc_PI"	%MD38	Floating-point nu...	5.0	5.0			
11	"Ki_step"	%MD42	Floating-point nu...	0.48	0.48			
12		<Add new>						

PLC Siemens

PLC_1 [CPU 1214C DC/DC/DC]

SIEMENS

RUN / STOP STOP

ERROR PAUSE

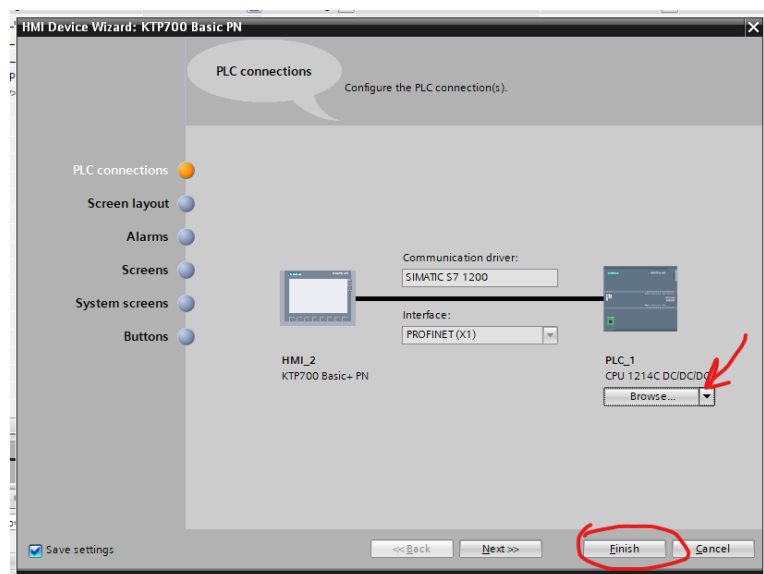
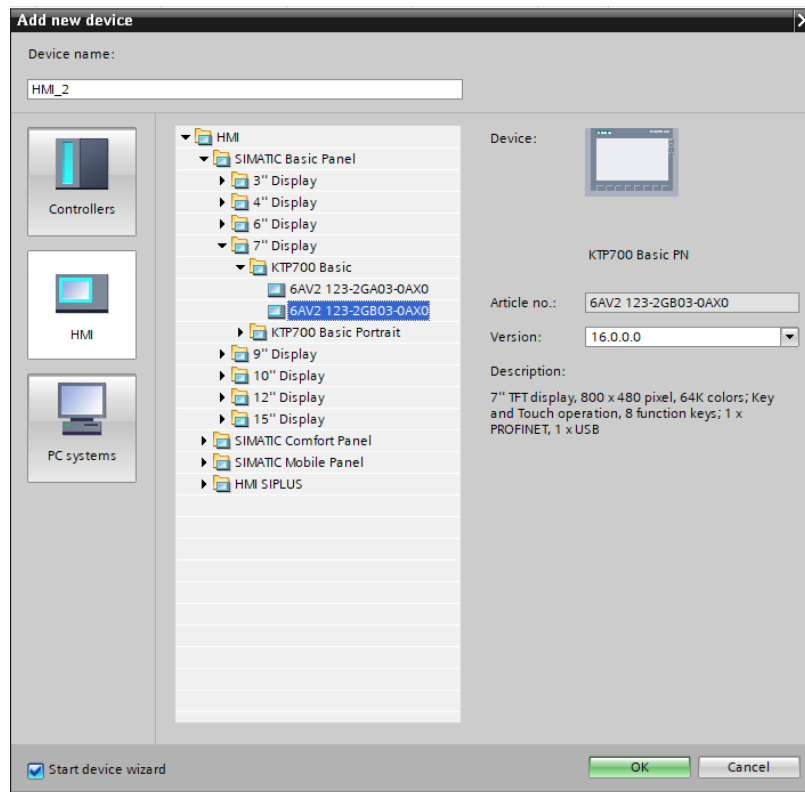
MAINT MRES

X1 192.168.0.1

<no project>

5. КРЕИРАЊЕ И КОНФИГУРАЦИЈА HMI ЕКРАНА

За потребе визуелизације и директног управљања процесом у реалном времену, потребно је да направимо HMI екран (Root screen) у оквиру WinCC Runtime/Basic HMI окружења. За сврху овог задатка одабран је KTP 700 Basic PN, чија се имплементација у TIA portal врши одабиром функције Add new device са врха Project tree-ја. Након избора панела, треба да се одабере наш PLC са падајуће листе чиме се остварује конекција између ова два дела хардвера.

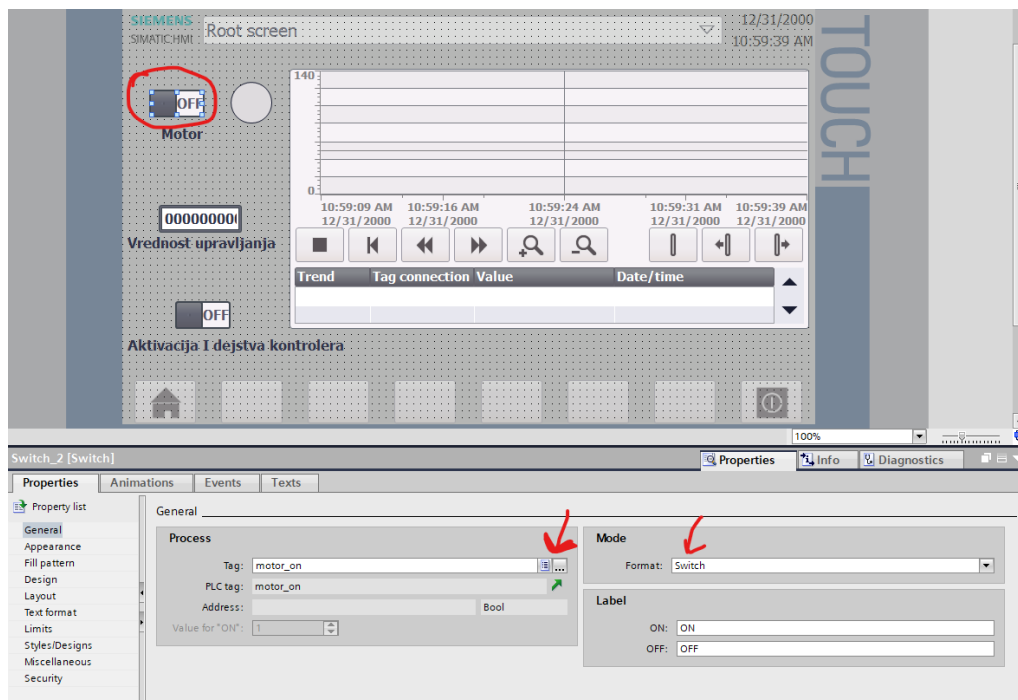


Едитовање HMI панела се врши на делу Project tree-ја под називом Screens, па по дифолту Root Screen, или било који други скрин који додамо касније ако нам је потребан.

На сам интерфејс HMI-а потребно је додати неколико ствари:

- Дугме које ће покретати и заустављати мотор, односно СТАРТ/СТОП дугме,
- Сијалицу која ће мењати боју у односу на то да ли је мотор упаљен или не,
- Поље у које можемо укуцавати жељену вредност управљања,
- Дугме које ће пуштати у рад И дејство ПИ контролера,
- И сам график са ког ћемо читавати вредности излазног сигнала регулатора, излазне вредности брзине мотора (у процентима) и временски интервал односно брзину реаговања система.

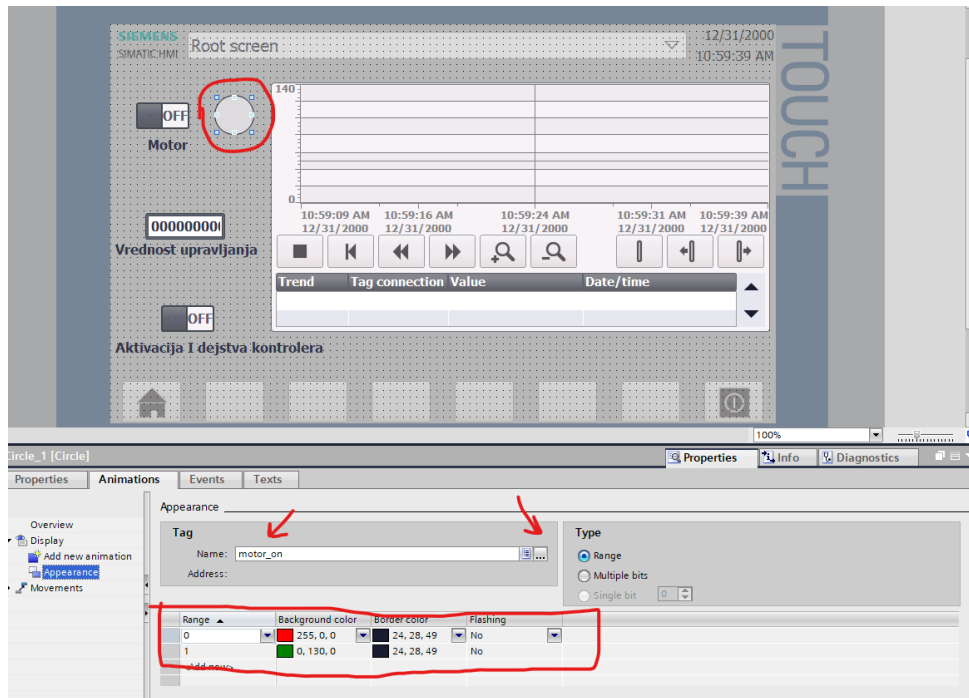
Ако кренемо од СТАРТ/СТОП дугмета, његова имплементација је прилично једноставна, из Toolbox-а са десне стране превучемо из секције Elements, прекидач односно Switch на панел. Кликом на тај прекидач и селектовањем Properties са доњег менија можемо едитовати понашање прекидача.



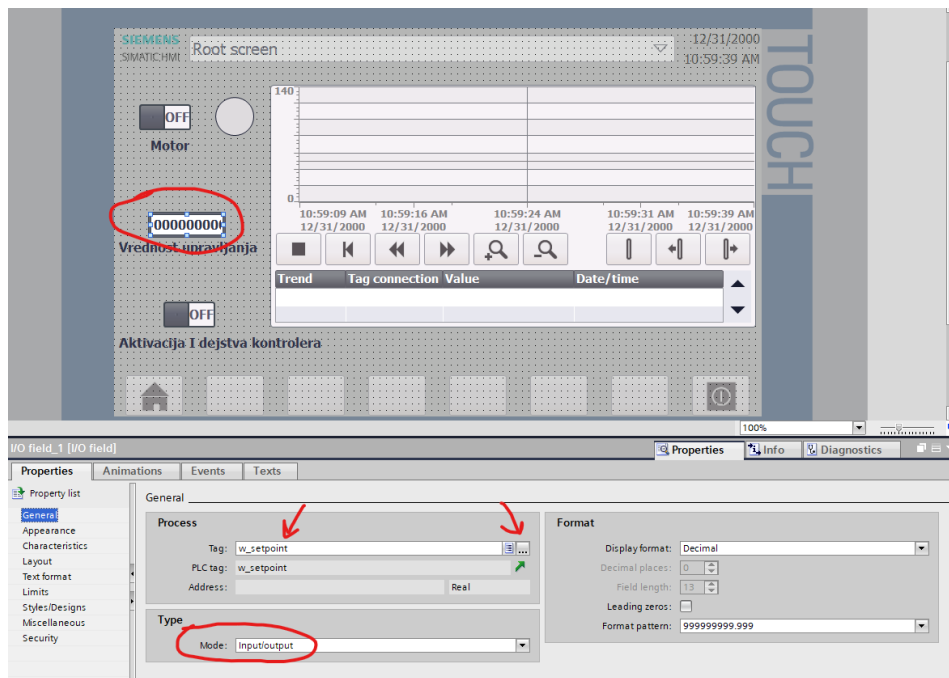
Довољно је да подесимо мод на Switch, и повежемо таг са тагом директно из PLC табеле тагова, TIA Portal ће аутоматски креирати одговарајући HMI таг у табели тагова за HMI са одговарајућом конекцијом са одговарајућим PLC-ом.

Како би убацили лампицу која мења боју у зависности од статуса мотора, и Toolbox-а на панел превлачимо из дела Basic objects – Circle. Пошто је статус мотора једноставна бинарна промена јединица и нула, у делу Animations од селектованог круга можемо изабрати Display -> Appearance и повезати понашање сијалице и статуса мотора поновно бирајући таг motor_on, а затим испод тога дефинисати боје сијалице за одговарајуће модове мотора. У мом случају зелено за рад мотора, а црвено док је искључен.

Пројектни задатак – аутоматско управљање

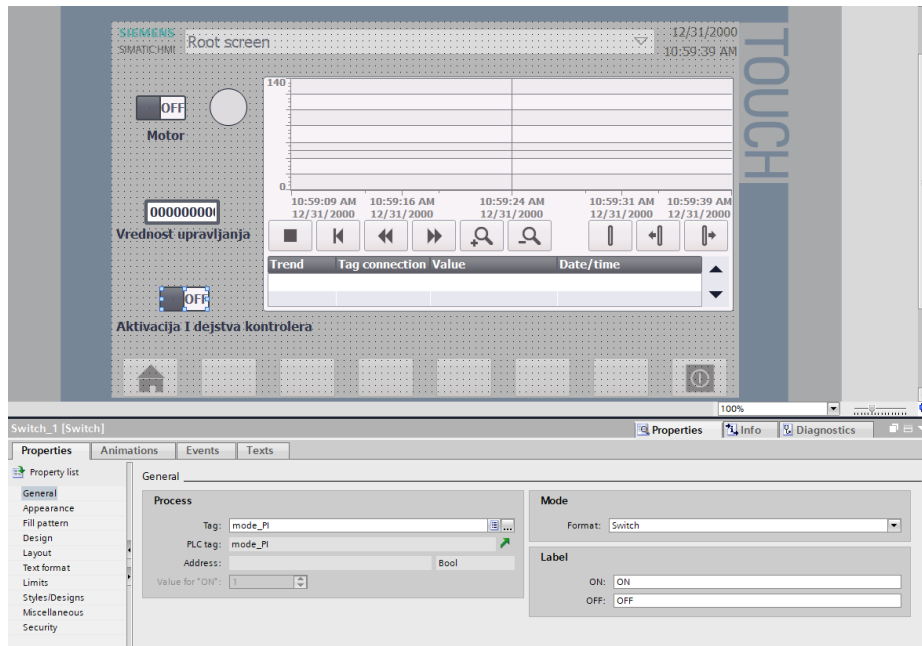


За поље у које можемо да уносимо вредност управљања која нам је потребна из Toolbox-а са десне стране из одељка Elements бирамо I/O Field, и превлачимо га на наш панел. Затим у менију Properties ово поље повезујемо са одговарајућим тагом и дефинишемо децимални формат унешеног броја.

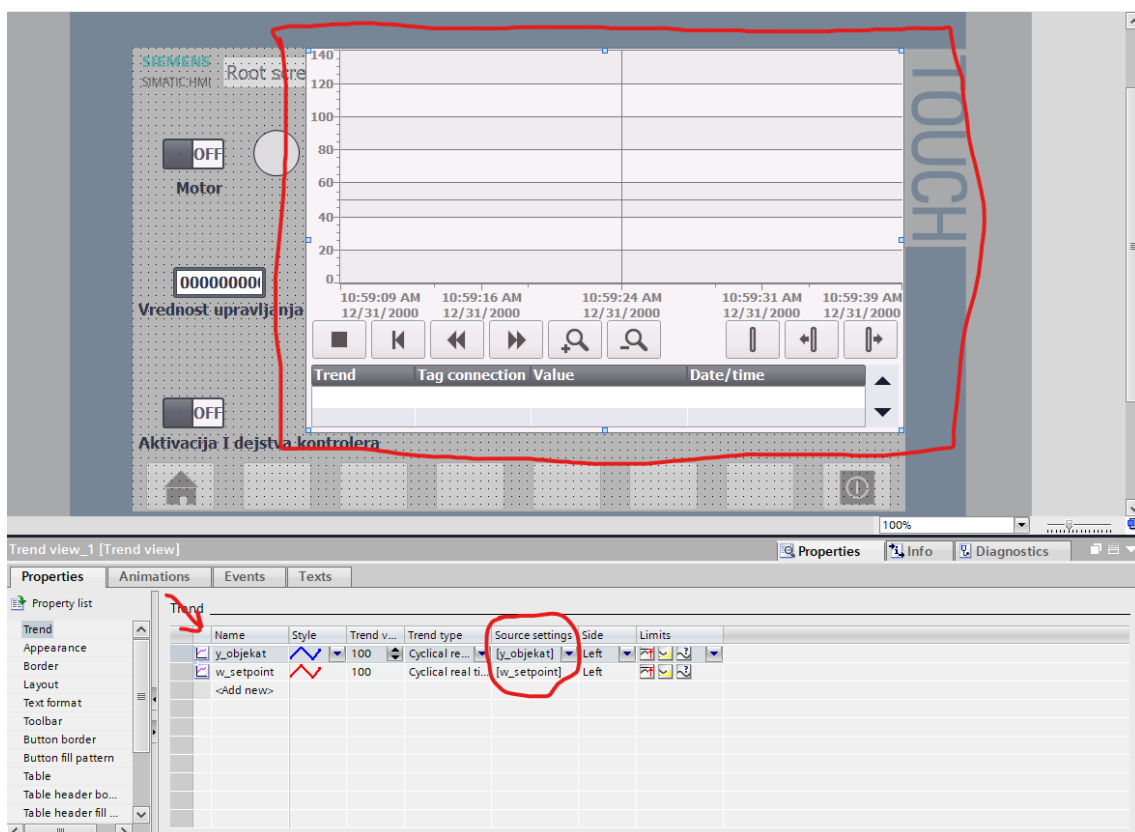


За прекидач који контролише дејство И користимо исти принцип као код креирања дугмета за СТАРТ/СТОП, као на слици:

Проектни задатак – аутоматско управљање



За убацивање самог графика користимо из Toolbox-а елемент назван Trend View (или f(x) Trend View ако је коришћен Comfort панел при креирању HMI-а), на следећи начин:

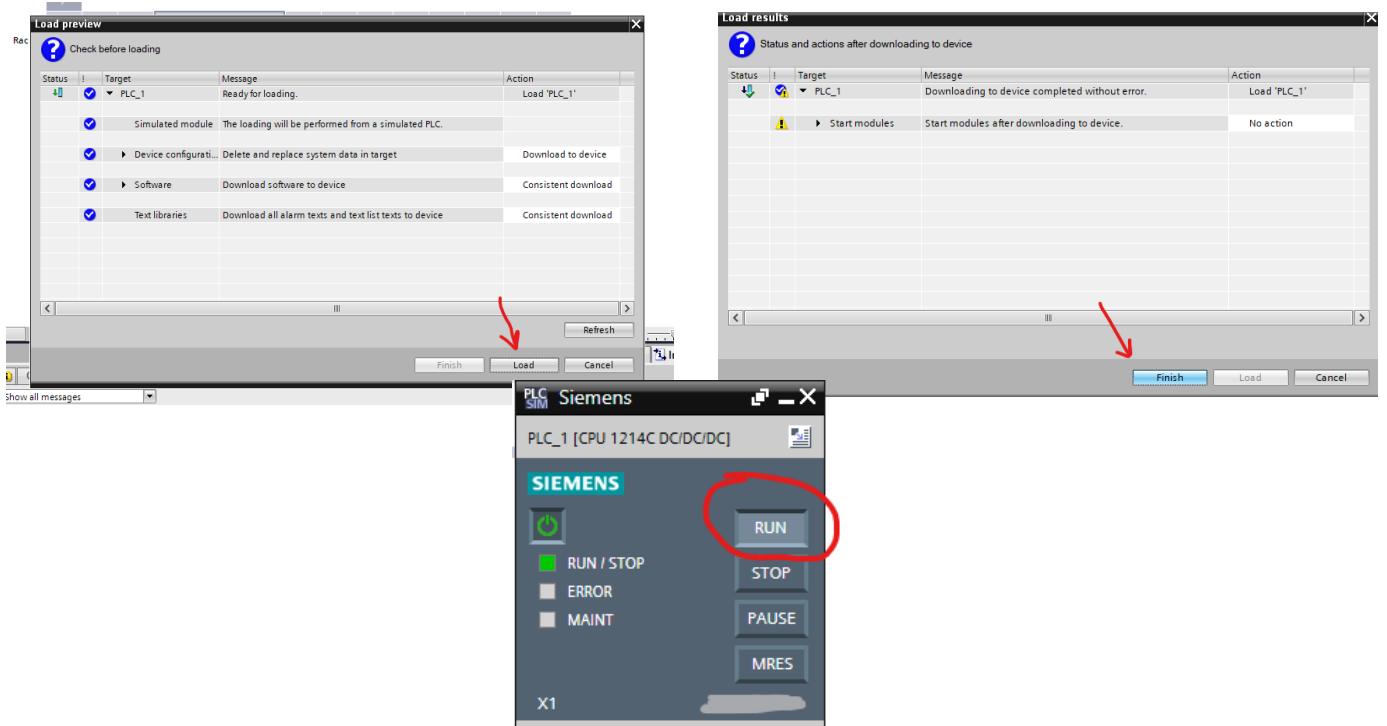
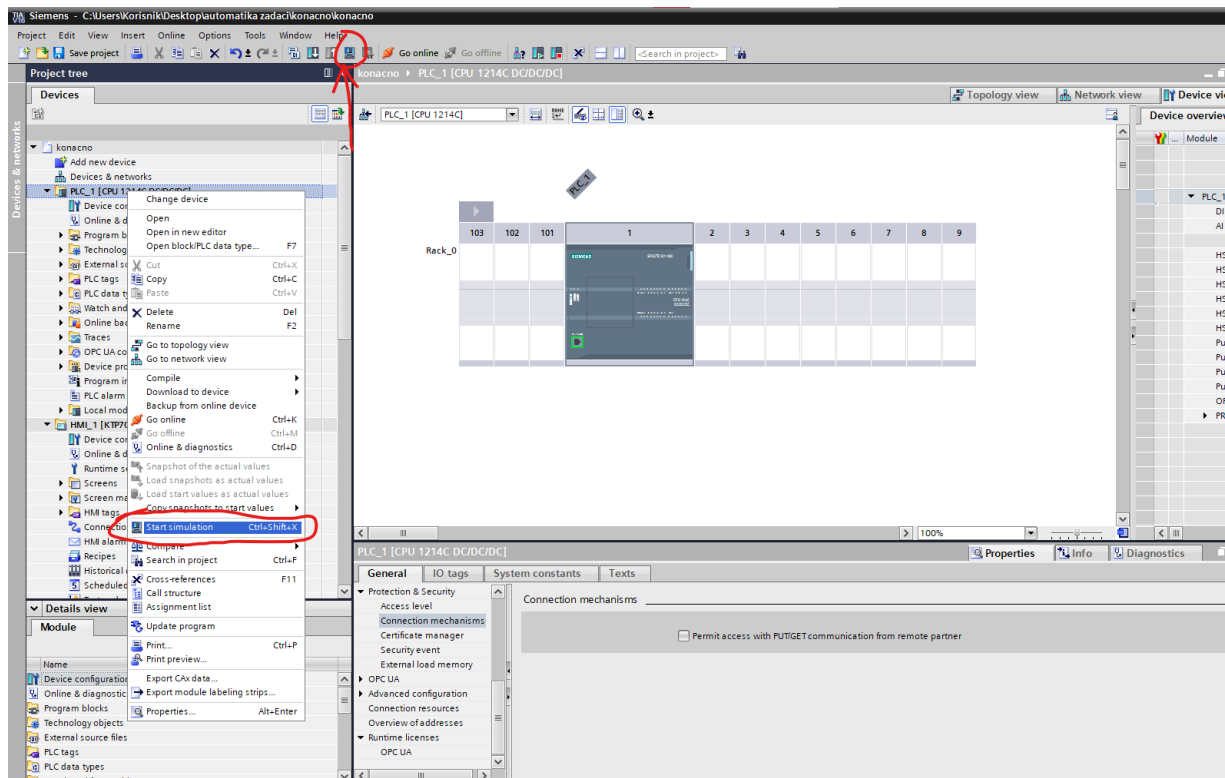


Потребно је подесити Trend type да буде Cyclical real time, како би пратио циклично извршавање кода на сваких 0,04 [s]. Убацујемо две линије, једна која представља сигнал контролера, а друга, у овом случају плава представља реалну брзину коју за тај сигнал постиже мотор, изражену у процентима.

6. ПОКРЕТАЊЕ СИМУЛАЦИЈЕ И ДИСКУТОВАЊЕ РЕЗУЛТАТА

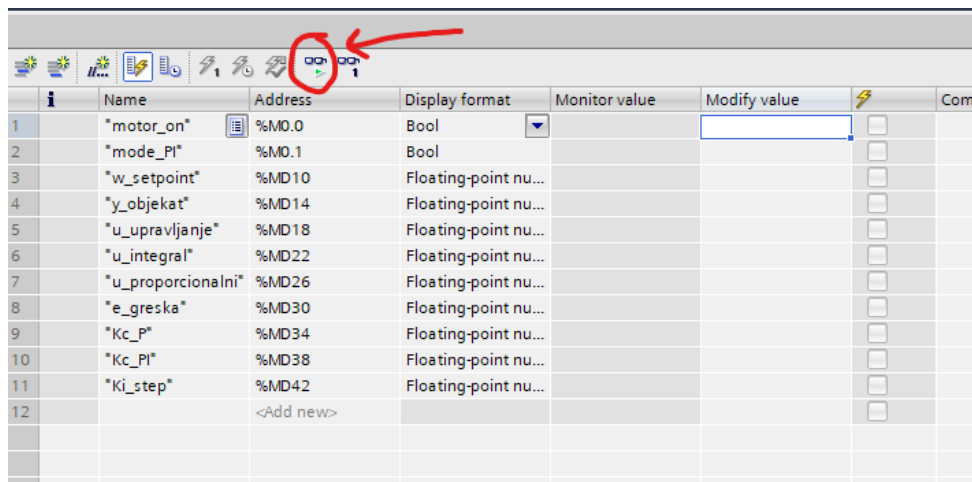
Након завршеног подешавања HMI панела, можемо покренути симулацију и погледати резултате.

Покретање симулације у TIA Portalu се врши у више корака. Прво је потребно да покренемо симулацију самог PLC дела, као на сликама:



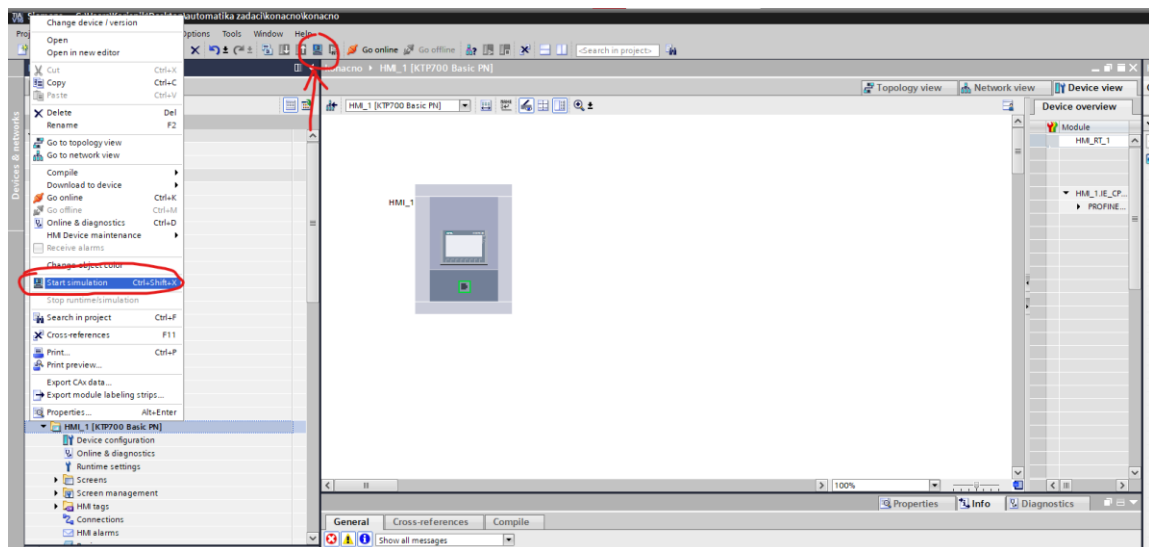
Проектни задатак – аутоматско управљање

Затим покрећемо мониторинг вредности у Watch table-у, као на слици:



	Name	Address	Display format	Monitor value	Modify value		Comments
1	"motor_on"	%M0.0	Bool			☐	
2	"mode_PI"	%MD1	Bool			☐	
3	"w_setpoint"	%MD10	Floating-point nu...			☐	
4	"y_objekat"	%MD14	Floating-point nu...			☐	
5	"u_ustrojanje"	%MD18	Floating-point nu...			☐	
6	"u_integral"	%MD22	Floating-point nu...			☐	
7	"u_proporcionalni"	%MD26	Floating-point nu...			☐	
8	"e_greska"	%MD30	Floating-point nu...			☐	
9	"Kc_P"	%MD34	Floating-point nu...			☐	
10	"Kc_PI"	%MD38	Floating-point nu...			☐	
11	"Ki_step"	%MD42	Floating-point nu...			☐	
12		<Add new>				☐	

Затим покрећемо симулацију HMI-а на исти начин као и PLC, односно:



Када се заврши компајлирање и отвори симулирани HMI, спремни смо за симулације.

Како би смо упоредили управљање постигнуто П и ПИ контролером, спровешћемо три симулације, прва када је модел под чистим П режимом са пропорционалним појачањем $K_{c_P} = 2.0$, друга, када је модел такође под чистим П режимом рада али већим појачањем $K_{c_P} = 5.0$, а затим како се модел понаша при ПИ режиму управљања.

6.1. Прва симулација: Чист П режим са $K_{c_P} = 2.0$

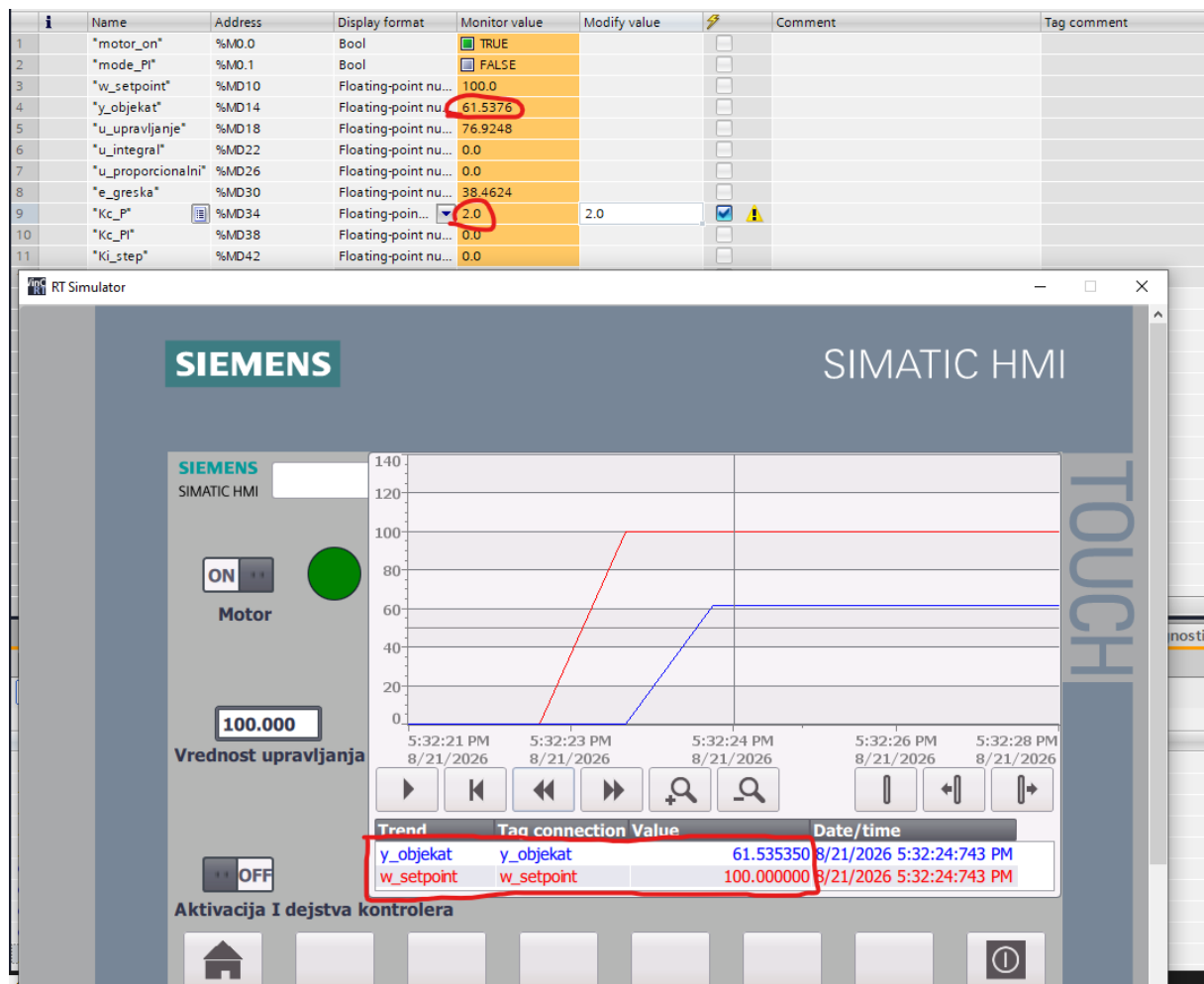
- Поставки параметара у Watch table-у: $mode_PI = FALSE$, $K_{c_P} = 2.0$, $w_setpoint = 100.0$

Излаз система $y_objekat$ асимптотски тежи вредности од приближно 61.5%, односно формира се значајна статичка грешка у стационарном стању $e_{st} = w - y = 100\% - 61.5\% = 38.5\%$.

У чистом П режиму, управљачки сигнал је директно пропорционалан грешки ($u = K_{c_P} \cdot e$). Пошто је за одржавање стационарног стања објекта потребан одређени напон, односно управљање u , грешка e не може бити једнака нули (ако би $e = 0$, онда би и $u = 0$, па би излаз пао). Статичка грешка за систем са $G(s) = K/(\tau s + 1)$ рачуна се по формули:

$$e_{st} = \frac{1}{1 + K \cdot K_{c_P}} = \frac{1}{1 + 0.8 \cdot 2.0} = \frac{1}{2.6} \approx 38.46\%$$

што се савршено поклапа са резултатом из симулације, што се види са слике:



6.2. Друга симулација: Чист П режим са повећаним појачањем $K_{c_P} = 5.0$

- Поставки параметара у Watch табели: $mode_PI = FALSE$, $K_{c_P} = 5.0$, $w_setpoint = 100.0$

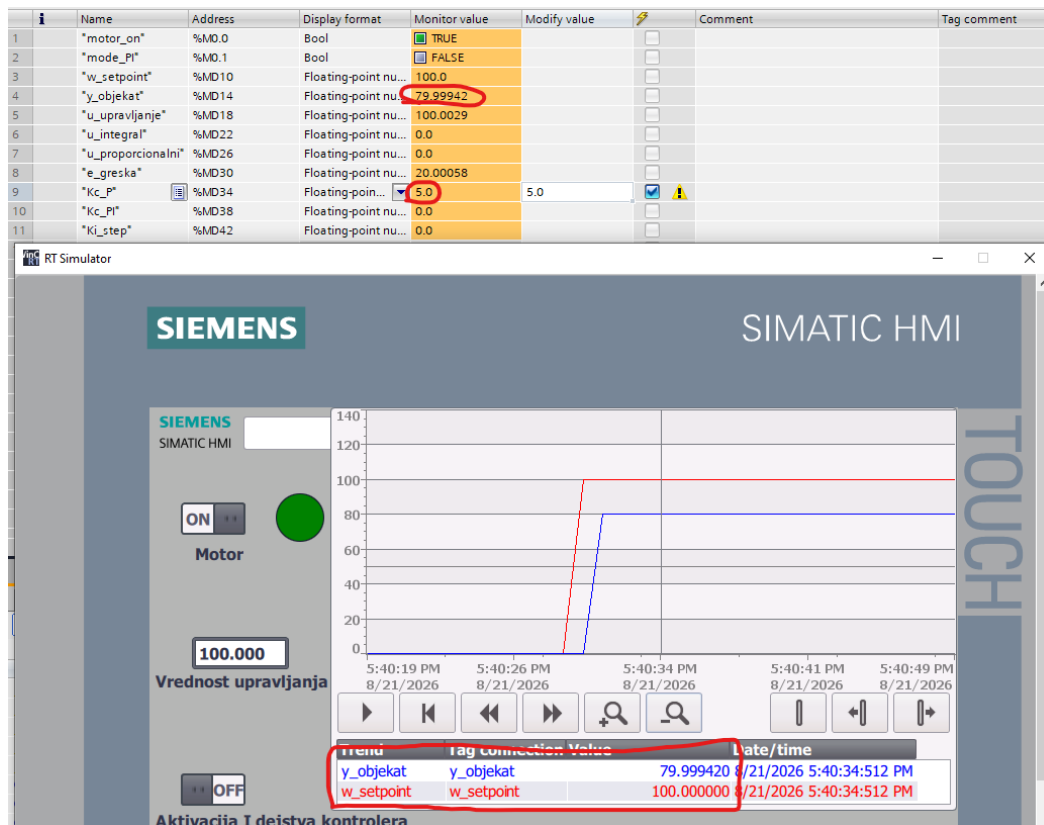
Излаз система $y_objekat$ брже расте и у стационарном стању достиже вредност од 80.0%. Статичка грешка је смањена, али и даље присутна: $e_{st} = 100\% - 80\% = 20.0\%$.

Повећањем пропорционалног појачања са 2.0 на 5.0, систем постаје агресивнији (јавља се бржи одзив), а статичка грешка се смањује у складу са теоријом:

$$e_{st} = \frac{1}{1 + 0.8 \cdot 5.0} = \frac{1}{5.0} = 20.0\%$$

Овај експеримент доказује да се повећањем K_c у П режиму статичка грешка може смањити, али се никада не може потпуно елиминисати без увођења интегралног дејства.

Ови резултати се поклапају са резултатима симулације:



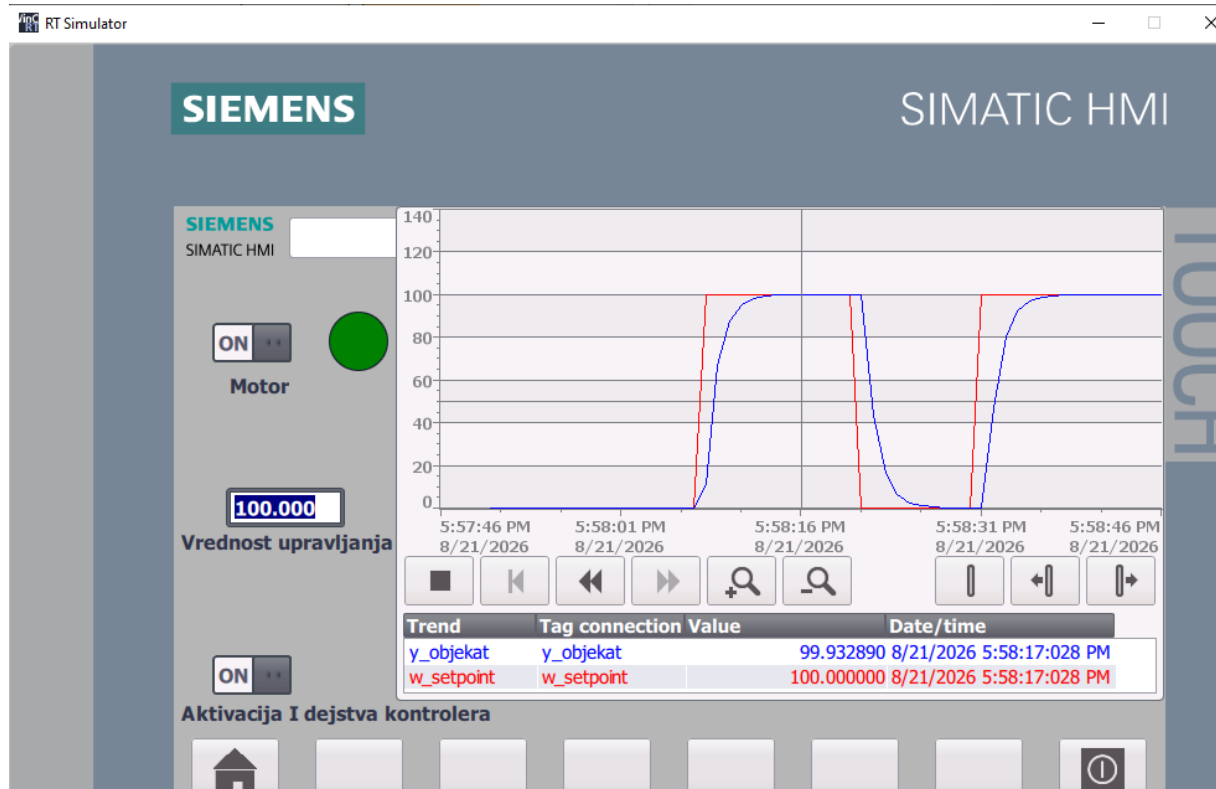
6.3. Трећа симулација: ПИ режим са $K_{c_PI} = 3.0$ и $K_{i_step} = 0.48$ ($K_i = 12.0 [s^{-1}]$)

- Поставки параметара у Watch табели: mode_PI = TRUE, $K_{c_PI} = 3.0$, $K_{i_step} = 0.48$, w_setpoint = 100.0

Излаз система y_objekat постепено али сигурно расте све док се у потпуности не изједначи са задатом вредношћу од 100.0%, или било којом другом вредношћу управљања коју му задамо, докле год она улази у физички упсег могућег извршавања (до 100 %). Статичка грешка у стационарном стању износи $e_{st} = 0.0$.

Увођењем интегралне компоненте, регулатор започиње акумулирање малих одступања кроз време (u_integral). Чак и када пропорционални део почне да опада како се у приближава w, интегрални део задржава неопходан степен управљања и који држи систем на 100%. Захваљујући имплементираном Anti-Reset Windup (ARW) систему

(ограничење интегратора на опсег од 0.0 до 125.0), спречено је прекомерно нагомилавање интегралне суме током прелазног процеса, чиме је остварен миран прелаз у стационарно стање без прекорачења (препуњавања интегратора) и без осцилација. Симулација потврђује теорију и изгледа овако:



7. ПОРЕЂЕЊЕ РЕЗУЛТАТА: MATLAB/SIMULINK VS. PLC (TIA PORTAL)

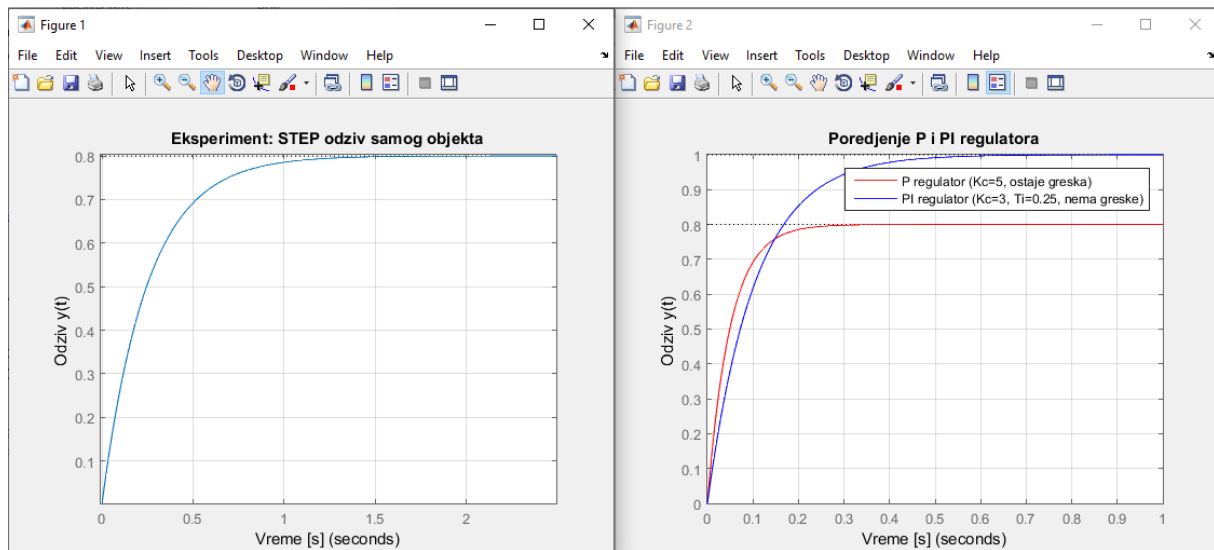
7.1. Симулација у MATLAB-у

Да би се извршила верификација тачности употребљеног SCL кода и дискретног модела у TIA Portal-у, спроведено је поређење добијених одзива са теоријским симулационим моделом изграђеним у програму MATLAB / Simulink.

У MATLAB-у је коришћена следећа скрипта како би се добили потребни графици:

```
Kp = 0.8;
T = 0.25;
Ts = 0.04;
G = tf(Kp, [T 1]);
figure(1);
step(G);
title('Eksperiment: STEP odziv samog objekta');
xlabel('Vreme [s]'); ylabel('Odziv y(t)');
grid on;
Kc_p = 5.0;
W_p = feedback(Kc_p * G, 1);
Kc_pi = 3.0;
Ti = 0.25;
C_pi = tf(Kc_pi * [Ti 1], [Ti 0]);
W_pi = feedback(C_pi * G, 1);
figure(2);
step(W_p, 'r', W_pi, 'b');
title('Poredjenje P i PI regulatora');
legend('P regulator (Kc=5, ostaje greska)', 'PI regulator (Kc=3, Ti=0.25, nema greske)');
xlabel('Vreme [s]'); ylabel('Odziv y(t)');
grid on;
```

Када покренемо скрипту добијамо следећа два дијаграма:



- Одзив самог објекта у отвореној петљи (figure(1))

Анализира се одзив објекта $G(s) = 0.8/(0.25s + 1)$ на одскачну побуду без присутности регулатора. Резултат који добијамо јесте да излаз $y(t)$ експоненцијално расте, и у стационарном стању достиже вредност једнаку статичком појачању објекта $y_{st} = K_p = 0.8$ (80%). Време смирења износи $t_s = 3\tau = 0.75$ [s].

- Поређење П и ПИ регулатора у затвореној петљи (figure(2))

За П регулатор (црвена линија W_p), при $K_{c_p} = 5.0$, преносна функција затвореног кола износи:

$$W_p(s) = \frac{5 \cdot G(s)}{1 + 5 \cdot G(s)} = \frac{4}{0.25s + 5} = \frac{0.8}{0.05s + 1}$$

Стационарна вредност је $y_{st} = 0.8$ (80%), што значи да у затвореном колу остаје статичка грешка од 20% ($e_{st} = 1 - 0.8 = 0.2$). Одзив је бржи у односу на отворену петљу ($\tau_{zk} = 0.05$ s), али грешка остаје.

За ПИ регулатор (плава линија W_{pi}): Применом ПИ регулатора са $K_{c_pi} = 3.0$ и $T_i = 0.25$ s остварује се поништавање доминантног пола објекта ($T_i = T = 0.25s$). Преносна функција регулатора је:

$$C_{pi}(s) = 3 \cdot \frac{0.25s+1}{0.25s}$$

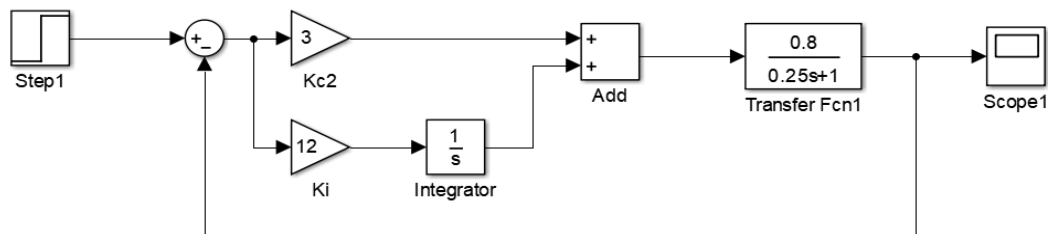
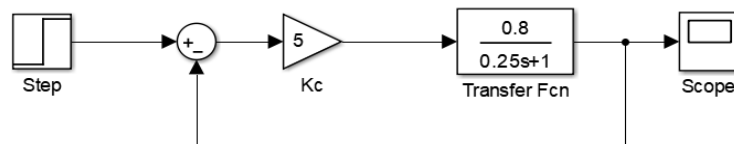
Преносна функција затвореног кола постаје:

$$W_{pi}(s) = \frac{\frac{2.4}{0.25s}}{1 + \frac{2.4}{0.25s}} = \frac{1}{0.104s + 1}$$

Излаз $y(t)$ асимптотски достиже тачно 1.0 (100%), чиме је статичка грешка потпуно елиминисана ($e_{st} = 0$), без појаве прекорачења (препуњавања интегратора).

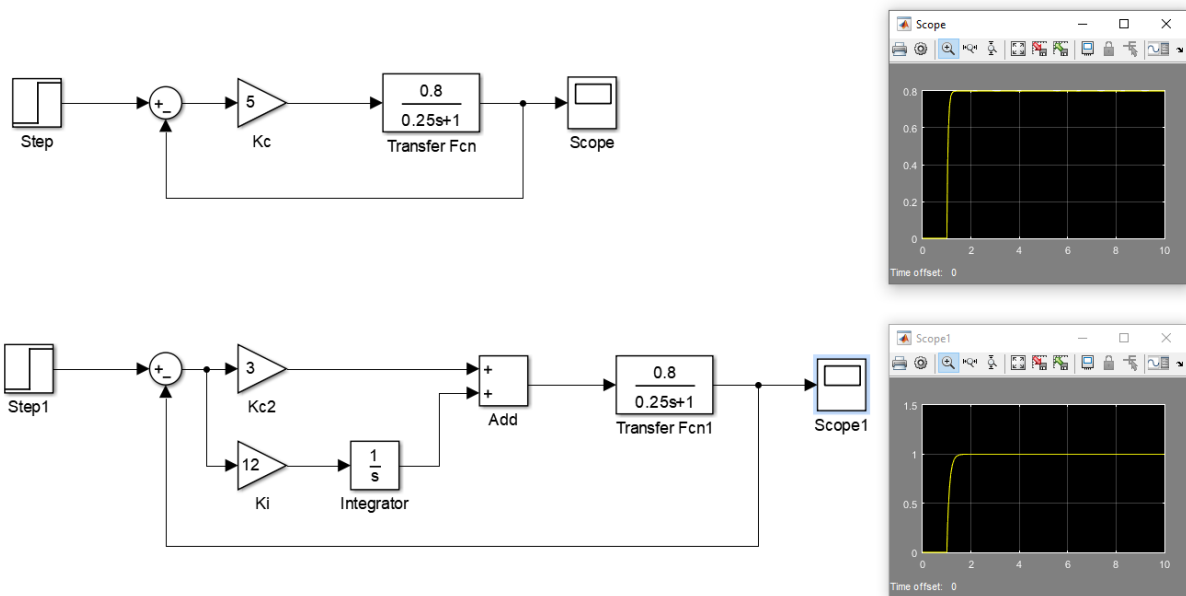
7.2. Симулација у SIMULINK-у

Модел се састоји од два одвојена подсистема који паралелно анализирају одзив објекта $G(s) = 0.8/(0.25s + 1)$ на јединичну одскочну побуду (Step = 1.0).



- Горњи подсистем – Чист П регулатор ($K_c = 5$):
 1. Step: Улазни сигнал задате вредности $w(t) = 1.0$
 2. Sum блок (+-): Формира сигнал грешке $e(t) = w(t) - y(t)$.
 3. K_c (Gain блок = 5): Пропорционално појачање регулатора.
 4. Transfer Fcn ($0.8/(0.25s+1)$): Континуални модел објекта управљања.
 5. Scope: Блок за снимање и приказ одзива излазне величине $y(t)$.
- Доњи подсистем – Паралелни ПИ регулатор ($K_{c2} = 3$, $K_i = 12$):
 1. Step1: Улазни сигнал задате вредности $w(t) = 1.0$
 2. Sum блок (+-): Формира сигнал грешке $e(t) = w(t) - y(t)$
 3. Пропорционална грана ($K_{c2} = 3$): Појачава сигнал грешке са коефицијентом $K_c = 3$
 4. Интегрална грана ($K_i = 12$ и Integrator $1/s$): Формира интегрално дејство $K_i e(t) dt$, где је континуално интегрално појачање $K_i = 12.0 \text{ s}^{-1}$ (што директно одговара дискретном кораку $K_{i_step} = 0.48$ за $T = 0.04 \text{ s}$ јер је $K_i * T = 12 * 0.04 = 0.48$)
 5. Add блок: Сабира пропорционалну и интегралну компоненту у укупни управљачки сигнал $u(t) = P(t) + I(t)$
 6. Transfer Fcn1 ($0.8/(0.25s+1)$): Модел објекта
 7. Scope1: Блок за праћење одзива ПИ регулатора.

Дуплим кликом на два Scope-а добијамо следеће дијаграме:



Приказ са Scope-а (за П регулатор) потврђује горенаведене ставка, да систем са пропорционалним појачањем $K_c = 5$ достиже стационарну вредност $y_{st} = 0.8$. Грешка од 20% остаје трајно присутна јер управљачки сигнал опада како се излаз приближава задатој вредности, па нема елемента који би додатно погурао овај излаз до 1.0.

Приказ са Scope-а 1 (за ПИ регулатор) нам потврђује да интегратор у доњој грани успешно акумулира грешку кроз време све док је $e(t)$ различито од 0. Због тога излаз асимптотски али брзо достиже пуну вредност $y_{st} = 1.0$, а статичка грешка се потпуно елиминише ($e_{st} = 0$).

8. ЗАКЉУЧАК

У овом раду успешно је пројектован, имплементиран и анализиран дискретни систем аутоматског управљања за систем првог реда преносне функције $G(s) = 0.8/(0.25s + 1)$. Симулација и валидација алгоритма извршене су коришћењем програмибилног логичког контролера Siemens S7-1214 DC/DC/DC у окружењу TIA Portal, као и рачунарских пакета MATLAB и Simulink. На основу спроведеног истраживања и експерименталних резултата, могу се дефинисати следећи кључни закључци.

Дискретизација континуалног модела Ојлеровом методом и његова имплементација у SCL језику унутар цикличног прекидног блока OB30 ($T = 40 \text{ ms}$) показала је велику прецизност. Одзиви добијени на PLC-у у потпуности одговарају теоријским одзивима из MATLAB/Simulink симулација.

Примена чистог пропорционалног регулатора ($K_c = 2.0$ и $K_c = 5.0$) потврдила је основни теоријски недостатак овог типа управљања — карактеристичну статичку грешку (38.5% односно 20.0%). Иако повећање појачања K_c убрзава одзив и смањује грешку, она се не може потпуно елиминисати без интегралног дејства. Увођењем интегралне компоненте ($K_{c_PI} = 3.0$, $K_{i_step} = 0.48$) статичка грешка је потпуно елиминисана ($e_{st} = 0\%$), а излазна величина је асимптотски достигла задату вредност без појава прекорачења или осцилација.

Имплементација ARW алгоритма са засићењем интегратора унутар SCL кода спречила је прекомерно нагомилавање интегралне суме током великих одступања, чиме је обезбеђена стабилност и адекватан прелазни процес при симулацији реалних ограничења излаза. Развијени HMI интерфејс (Root screen) са Trend View графиком омогућио је интуитивно праћење процеса у реалном времену, дијагностику и динамичку измену параметара путем Watch табеле. Овим је потврђено да креирани SCL код за Siemens S7-1200 представља поуздану, математички тачну и практично применљиву основу за управљање реалним индустријским процесима првог реда.